game-theory-layer4-integration

Game Theory Layer 4 Integration: Strategy & Pragmatics for the Debate Engine

Author: CL.Investigate1 (Computational Linguist) **Date:** 2026-05-21 **Status:** Proposal — pending implementation review **Origin:** Jeffrey Snover's Layer 4 (Strategy & Pragmatics) framework email, 2026-05-21

---

1. Executive Summary

Jeffrey Snover's Layer 4 framework decomposes strategic reasoning into three components:

1. **Utility Function** — quantitative scoring of argument graph state (the "Desire" in BDI)
2. **Opponent Model** — beliefs about other agents' knowledge, commitments, and strategy type
3. **Search & Decision Engine** — move selection that maximizes expected utility (the "Intention")

This document maps the framework against the current AI Triad debate engine, identifies what we already have, what's missing, and proposes five concrete implementation opportunities ordered by effort and value.

Key finding: The debate engine already has partial Layer 4 elements (commitment tracking, strategic hints, 10 canonical moves, QBAF-based evaluation). The highest-value additions are explicit utility functions for calibration and opponent-aware hint enrichment — both achievable without architectural overhaul.

---

2. Current Architecture Mapping

2.1 What the Engine Already Has

Layer 4 Component	Current Implementation	Location
Utility (implicit)	Phase-specific persona prompts + doctrinal boundaries	`lib/debate/prompts.ts`
Opponent Model (partial)	Commitment stores: asserted / conceded / challenged per speaker	`lib/debate/debateRunner.ts`
Move Vocabulary	10 canonical dialectical schemes (DISTINGUISH, COUNTEREXAMPLE, CONCEDE-AND-PIVOT, REFRAME, EMPIRICAL CHALLENGE,	`lib/debate/prompts.ts`

Layer 4 Component	Current Implementation	Location
	EXTEND, UNDERCUT, SPECIFY, INTEGRATE, BURDEN-SHIFT)	
Strategic Hints	Unanswered claims (every 3 rounds), SPECIFY opportunity detection, QBAF-grounded concession candidates	`lib/debate/prompts.ts`
Post-hoc Evaluation	15 calibration metrics, QBAF argument strength, 6 saturation + 5 convergence signals	`lib/debate/calibrationLogger.ts`, `lib/debate/phaseTransitions.ts`
Game Environment	Full argument network (nodes + edges), visible to all agents	`lib/debate/argumentNetwork.ts`

2.2 What's Missing

Layer 4 Component	Gap
Explicit Utility Function	Agents don't optimize a numeric payoff. Moves are prompt-guided, not utility-maximized. No way to evaluate "was this move strategically optimal?"
Opponent Strategy Model	Agents see opponent commitments but don't reason about opponent knowledge gaps, strategy type (cooperative vs. adversarial), or predicted next moves.
Search / Lookahead	No game tree, no minimax, no move comparison. Move selection is single-shot LLM generation.
Exploit Detection	No defenses against filibustering (weak claim flooding), dialectical drift (scope gerrymandering), or preference faking (strategic concession).
Adaptive Context Injection	Situation nodes injected statically based on topic similarity, not dynamically based on emerging cruxes.

2.3 Information Model

The debate operates under **perfect information with asymmetric emphasis**:

- **Perfect information:** The full argument network, all commitment stores, and all calibration signals are visible to all agents and the engine.
- **Asymmetric emphasis:** Each agent receives POV-filtered taxonomy context — Prometheus sees accelerationist nodes as primary tier, Sentinel sees safetyist nodes. This creates *de facto* knowledge asymmetry without formal hidden state.

Recommendation: Keep perfect information. Introducing hidden state (Poker model) would require Bayesian belief updating, bluffing mechanics, and information-disclosure as a move type — all of which add complexity disproportionate to value for a three-agent cooperative-adversarial research debate. The interesting strategic dynamics come from *attention allocation under budget constraints*, not from deception.

3. Five Implementation Opportunities

3.1 Explicit Utility Functions (Calibration Enhancement)

Effort: Low | **Value:** High | **Architecture change:** None — pure metric addition

Problem: We have no quantitative way to answer "did this agent make a good strategic move?" The 15 calibration metrics measure debate-level quality, not per-agent strategic effectiveness.

Proposal: Define a numeric utility function per agent, computed after each turn, that scores the argument network from that agent's perspective.

```
interface AgentUtility {
  /** Mean computed_strength of this agent's undefeated nodes */
  position_strength: number;

  /** Fraction of opponent nodes this agent has weakened below 0.3 */
  attack_effectiveness: number;

  /** Fraction of identified cruxes this agent has addressed */
  crux_engagement: number;

  /** Weighted composite — weights tunable per persona */
  composite: number;
}

// Persona-specific weighting
const UTILITY_WEIGHTS: Record<SpeakerId, {pos: number, atk: number, crux: number}> =
        {
  prometheus: { pos: 0.45, atk: 0.30, crux: 0.25 },  // Persuasion-leaning
  sentinel:   { pos: 0.30, atk: 0.25, crux: 0.45 },  // Evidence-leaning
  cassandra:  { pos: 0.20, atk: 0.25, crux: 0.55 },  // Truth-seeking-leaning
};
```

What this enables: - **Utility curves** per agent across rounds — detect stagnation (flat utility), runaway dominance (one agent's utility monotonically rising), or disengagement (utility declining without opponent pressure). - **Move quality scoring** — compare utility delta (Δu) per turn. Turns with Δu ≈ 0 despite available cruxes indicate strategic failure. - **Jeffrey's cooperative vs. adversarial distinction** — agents with high `crux_engagement` relative to `attack_effectiveness` are truth-seeking; the inverse indicates persuasion mode.

Integration point: Add `computeAgentUtility()` to `calibrationLogger.ts`. Log per-turn per-agent utility alongside existing metrics. No changes to prompts or debate flow.

3.2 Opponent-Aware Strategic Hints

Effort: Medium | **Value:** High | **Architecture change:** Extends existing hint system in `prompts.ts`

Problem: Current strategic hints (unanswered claims, SPECIFY opportunities, concession candidates) are game-state observations. They don't reason about *why* the opponent made certain moves or *where* the

opponent is vulnerable.

Proposal: Three new hint types, all computable from existing data structures:

A. Commitment Trap Detection

Scan the opponent's commitment store for internal tensions. If Agent X has conceded claim A but later asserts claim B that contradicts or is in tension with A, surface it:

```
COMMITMENT TENSION: Sentinel conceded "regulation alone cannot prevent misuse"
(round 3) but now argues "mandatory licensing will prevent misuse" (round 7).
Consider a DISTINGUISH move to probe whether Sentinel has refined their
position or is contradicting themselves.
```

Data source: Commitment stores (asserted + conceded) + argument network edges of type `attacks` between an agent's own nodes.

B. Capability Steering

Track which taxonomy branches each agent has drawn from most heavily (via `taxonomy_refs` on their nodes). Identify branches where the opponent has sparse coverage:

```
KNOWLEDGE GAP: Prometheus has cited acc-desires-* nodes 12 times but has
zero references to empirical methodology nodes (skp-beliefs-*).
Consider steering toward empirical validation claims where
Prometheus has less taxonomic grounding.
```

Data source: `taxonomy_refs` frequency distribution per speaker across the argument network.

C. Strategic Type Detection

Track the ratio of cooperative moves (EXTEND, INTEGRATE, CONCEDE-AND-PIVOT) vs. adversarial moves (COUNTEREXAMPLE, UNDERCUT, BURDEN-SHIFT) per agent across a rolling window:

```
STRATEGY SHIFT: Prometheus has moved from 70% adversarial moves (rounds 1-4)
to 40% adversarial (rounds 5-8). This may indicate genuine convergence
or rhetorical softening. Test with a SPECIFY move to confirm.
```

Data source: Dialectical scheme history on argument network edges.

3.3 Move-Quality Lookahead (One-Step)

Effort: High | **Value:** High | **Architecture change:** Adds evaluation step between draft generation and transcript commit

Problem: The LLM generates a single response with no comparison or quality gate. Low-value moves (restating existing positions, introducing tangential claims) are committed as-is.

Proposal: After the LLM generates a draft turn, run a lightweight strategic evaluation before committing:

```
Pipeline today:
  LLM draft → extract claims → commit to transcript → QBAF update

Pipeline with lookahead:
  LLM draft → extract claims → TENTATIVE network update → compute Δu
  if Δu < min_threshold:
    inject hint: "Your response mostly restated existing positions.
                  Try a different dialectical move."
    regenerate (1 retry max)
  else:
    commit to transcript → QBAF update
```

This is a **one-step lookahead** — one extra QBAF propagation per turn, no extra LLM call unless regeneration triggers. It addresses Jeffrey's "Search Engine" at minimum viable depth.

Constraint: Maximum 1 regeneration per turn to avoid infinite loops. If the retry also fails the threshold, commit it anyway and log a `low_utility_turn` calibration event.

Side benefit: Directly mitigates Jeffrey's "Filibustering" attack vector. Weak claims that don't shift utility get rejected before they pollute the graph.

3.4 Anti-Exploit Defenses

Effort: Medium | **Value:** Medium | **Architecture change:** Extends extraction pipeline and signal scoring

Jeffrey identifies three exploit patterns that autonomous agents would use. Even though our agents are LLM-driven (not adversarial optimizers), these patterns can emerge from prompt drift or degenerate debate states.

A. Anti-Filibustering: Claim Marginal Value Check

Current defense: Network GC at 175 nodes; duplicate detection at 30% word overlap. **Gap:** Weak claims below the duplicate threshold but above extraction confidence can flood the graph.

Fix: During claim extraction, reject nodes where:

```
base_strength < 0.25
  AND !isConnectedToCruxNode(node, cruxNodes)
  AND !isNovelScheme(node.edge?.scheme, recentSchemes)
```

This rejects low-value claims that neither engage cruxes nor introduce novel reasoning patterns.

B. Anti-Drift: Topic Coherence Signal

Current defense: `taxonomyRelevance.ts` scores node relevance at injection time. **Gap:** No runtime detection of agents steering the debate away from its cruxes.

Fix: Add a `topic_coherence` signal to the saturation scoring pipeline:

```
function computeTopicCoherence(
  recentClaims: ArgumentNetworkNode[],  // last 3 turns for this speaker
  cruxNodes: ArgumentNetworkNode[]
): number {
  // Mean embedding similarity between recent claims and crux centroid
  const cruxCentroid = meanEmbedding(cruxNodes.map(n => n.embedding));
  const similarities = recentClaims.map(c => cosineSimilarity(c.embedding,
        cruxCentroid));
  return mean(similarities);
}
```

Low coherence → inject a hint: *"Your recent arguments have drifted from the core disagreements. Re-engage with [crux description]."*

C. Anti-Preference-Faking: Concession Asymmetry Tracking

Current defense: Concession tracking in commitment stores. **Gap:** No mechanism to detect *strategic* concessions (conceding cheap nodes to extract valuable counter-concessions).

Fix: Track per-agent concession asymmetry:

```
const concessionValue = meanStrength(agent.conceded);
const attackValue = meanStrength(agent.activeAttackTargets);
const asymmetry = attackValue - concessionValue;
// High asymmetry = conceding weak nodes while pressing strong attacks
```

This is an **observation metric for calibration**, not a block. In truth-seeking debate, strategic concession patterns are informative — the metric tells us whether an agent is genuinely updating or gaming.

3.5 Adaptive Situation Injection

Effort: Low | **Value:** Medium | **Architecture change:** Extends `taxonomyRelevance.ts`

Problem: Situation nodes (`sit-*`) are injected based on static embedding similarity to the debate topic. A situation irrelevant at debate start may become highly relevant once a specific crux emerges.

Proposal: Re-score situation relevance at each phase transition:

```
function rescoreSituations(
  situations: SituationNode[],
  cruxNodes: ArgumentNetworkNode[],
  currentManifest: ContextInjectionManifest
): ScoredSituation[] {
  return situations.map(sit => {
    // Score against CURRENT cruxes, not just the original topic
    const cruxRelevance = maxSimilarity(sit.embedding, cruxNodes.map(n =>
        n.embedding));

    // Bonus for interpretive diversity (all 3 POVs disagree)
    const diversityBonus = sit.disagreement_types.size ≥ 2 ? 0.15 : 0;
```

```
    // Penalty for already-injected but never-referenced situations
    const wasInjected = currentManifest.situationNodeIds.includes(sit.id);
    const wasReferenced = argumentNetwork.nodes.some(n =>
        n.taxonomy_refs.includes(sit.id));
    const stalePenalty = wasInjected && !wasReferenced ? -0.20 : 0;

    return { ...sit, score: cruxRelevance + diversityBonus + stalePenalty };
  }).sort((a, b) => b.score - a.score);
}
```

Key behaviors: - Situations that match emerging cruxes rise to primary tier - Situations with maximum disagreement diversity (definitional + interpretive + structural) get a bonus — richest strategic terrain - Situations injected in prior phases but never referenced by any debater are demoted — `situation_crux_alignment` metric as a selection feedback signal

---

4. Implementation Order

Phase	Opportunity	Effort	Dependencies	Key Files
1	3.1 Utility Functions	Low	None	`calibrationLogger.ts`, `argumentNetwork.ts`
2	3.4 Anti-Exploit Defenses	Medium	3.1 (uses utility for marginal value)	`prompts.ts` (extraction), `phaseTransitions.ts` (coherence signal)
3	3.2 Opponent-Aware Hints	Medium	None (but better with 3.1 data)	`prompts.ts` (hint generation)
4	3.5 Adaptive Situation Injection	Low	None	`taxonomyRelevance.ts`, `taxonomyContext.ts`
5	3.3 Move-Quality Lookahead	High	3.1 (utility delta as gate)	`debateRunner.ts` (pipeline change)

Phase 1 is a pure calibration addition with no debate-flow changes. Phases 2-4 are independent and can be parallelized. Phase 5 depends on having a working utility function and is the most architecturally invasive.

---

5. What We Deliberately Defer

Full Minimax / Game Tree Search

The branching factor is too high (10 move types x variable claim counts x 3 agents). Jeffrey's RL/self-play suggestion requires a stable reward signal — build utility functions first (opportunity 3.1), then revisit RL as a future research direction.

Hidden State / Poker Model

Adds complexity without proportional value for our cooperative-adversarial hybrid. Our agents are *characters with known positions*, not deceptive actors. Asymmetric taxonomy emphasis already provides

the interesting strategic dynamics.

Preference Faking as a Feature

Jeffrey lists this as an attack vector. For truth-seeking research debates, we detect it (calibration metric 3.4C), we don't enable it.

Decentralized / Gas-Cost Utility

Jeffrey's resource-bounded utility (`P(Victory) x V(Victory) - Cost(Computation)`) applies to DAO/blockchain contexts. Our resource constraint is API token budget, which the engine already tracks via `api_calls_used` and budget multipliers in phase transitions. We don't need a formal cost-utility tradeoff — the existing budget gating is sufficient.

---

6. The Advisor's Question, Answered

For our debate engine: perfect information, resource-bounded.

The debate is transparent (all commitments visible, full argument network shared), but resource-bounded (API token budget, round limits, context window size). Strategic depth comes not from hiding information but from **choosing where to allocate finite attention** — which cruxes to engage, which opponent claims to address vs. let stand, when to concede vs. fight.

The math that matters is: - **QBAF propagation as a utility estimator** (existing, needs per-agent projection) - **Attention allocation under budget constraints** (existing budget gating, needs utility-aware move selection)

Both are buildable incrementally on the current architecture without moving to Bayesian game modeling.

---

7. References

- Jeffrey Snover, "Layer 4: Strategy & Pragmatics" (email, 2026-05-21)
- Dung, P.M. (1995). "On the acceptability of arguments and its fundamental role in nonmonotonic reasoning, logic programming and n-person games." *Artificial Intelligence*, 77(2), 321-357.
- Baroni, P., Rago, A., & Toni, F. (2019). "From fine-grained properties to broad principles for gradual argumentation." *International Journal of Approximate Reasoning*, 105, 252-286. [QBAF foundations]
- Walton, D. (2008). *Argumentation Schemes*. Cambridge University Press. [Canonical scheme taxonomy]
- Prakken, H. (2006). "Formal systems for persuasion dialogue." *The Knowledge Engineering Review*, 21(2), 163-188. [Dialogue protocol formalization]
- Silver, D., et al. (2017). "Mastering the game of Go without human knowledge." *Nature*, 550, 354-359. [Self-play RL precedent for Jeffrey's RL suggestion]

Appendix A: Mapping Jeffrey's Framework to BDI

Jeffrey's Component	BDI Category	Debate Engine Analog
Utility Function	Desire	`AgentUtility.composite` — what state of the argument graph does the agent want?
Opponent Model	Belief (about others)	Commitment stores + capability steering + type detection — what does the agent believe about opponents?
Search Engine	Intention	Hint-guided move selection + lookahead gate — what move does the agent intend to make?
Commitment Store	Belief (public)	`commitments.asserted / conceded / challenged` — already implemented
Capability Model	Belief (about knowledge)	`taxonomy_refs` frequency distribution — new, computable from existing data
Type/Persona	Belief (about strategy)	Cooperative/adversarial move ratio — new, computable from existing data
Dialectical Minimax	Intention (optimal)	One-step lookahead with utility delta gate — proposed
RL Self-Play	Intention (learned)	Deferred — requires stable reward signal from utility functions first

Appendix B: Attack Vector Defense Matrix

Attack Vector	Jeffrey's Description	Current Defense	Proposed Addition	Metric
Filibustering	Flood graph with weak claims	GC at 175 nodes, 30% overlap dedup	Claim marginal value check (base_strength + crux connection)	`low_value_claims_rejected` count
Dialectical Drift	Introduce tangential sub-arguments to exhaust opponent budget	Taxonomy relevance scoring at injection	Topic coherence signal (semantic drift from crux centroid)	`topic_coherence` per speaker per turn
Preference Faking	Concede cheap claims to extract valuable counter-concessions	Concession tracking	Concession asymmetry metric (mean conceded strength vs. mean attack target strength)	`concession_asymmetry` per agent